COHERENCE DEPTH AND AFFINE RECURRENCE IN A CUBICAL EXTENSION CALCULUS

HALVOR LANDE

e-mail address: hsl@awc.no

Abstract. We study framework extension over a fixed cubical object theory rather than over "cubical type theory" in the abstract. The theorem target is a cubical extension calculus built on a chosen CCHM-style cubical core with univalence and Kan composition, together with a discipline of globally acting opaque extensions whose public interfaces are counted only up to canonical telescope isomorphism. We call the least historical depth at which the resulting normalized obligation types stabilize up to canonical equivalence the coherence depth. Within this calculus we prove a payload-aware trace principle decomposing each sealed public interface into new core payload and resolved coherence trace, and we measure extension debt by the presentation-invariant minimal opaque cost μ. The cubical metatheory then yields a depth-two theorem stack: the normalized obligation sets stabilize at depth 2 up to canonical equivalence; separately, every primitive historical field of arity at least 3 is eliminable from a μ-minimal public presentation by cubical Kan operations; every surviving binary clause factors through the previous two normalized signatures; and a binary obstruction remains genuinely independent. Abstracting that proof pattern yields a universal affine recurrence for minimal opaque cost. In extensional systems with UIP the depth collapses to 1. In the fixed fully coupled cubical extension calculus studied here the depth is 2, so

$$\mu_{n+1} = \mu_n + \mu_{n-1} + 2c$$

under constant payload size c; after the shift $U_n = \mu_n + 2c$, the sequence is Fibonacci. The accompanying Cubical Agda development formalizes the candidate-and-obligation calculus, the cubical upper/lower metatheory, and the abstract recurrence package; section 6 records the exact paper-to-Agda map, the trusted boundary, and where the passage to the surface-syntactic quantity μ still uses the paper-level bridge.

1. Introduction

1.1. **The metatheoretic question.** This paper concerns a simple but surprisingly rigid question.

> When a type-theoretic core calculus is extended by sealing a new structure against the existing interface, how does the cost of integration scale with the historical depth of the library?

The question is not about transparent user developments inside a fixed library. It is about *sealed* extensions: new type formers, modalities, or promoted interface packages whose public behavior becomes part of the global exported interface. More specifically,

Key words and phrases: cubical type theory, extension calculus, coherence, univalence.

the paper studies the worst-case regime of *foundational core extensions*: globally acting additions whose exported payload must be specified across the currently active interface. Examples include extending a univalent universe hierarchy, where the decoding machinery must be updated against the library's active codes; adding a global modality presented by explicit modal dependent-type syntax, such as a Fitch-style dependent right adjoint or its cubical multimodal refinements [BCM+20, AKGB24], whose action must distribute across the exported type formers and transports already present; and heavily promoted generic interfaces whose action is intended to range over the whole library. In that setting, extension is constrained not only by local typing rules, but also by the obligation to interact coherently with the eliminators, transports, computation principles, and comparison data already exported by the library.

Orthogonal extensions are not the subject of this recurrence law. Adjoining a new datatype such as `Tree` to a library that already contains `List` and `Socket` need not introduce primitive `Tree`–`Socket` interaction data merely to preserve canonicity. Such sparse or local extensions are important in practice, but they do not exhibit the maximal coupling pattern that this paper isolates. Ordinary user-level library growth therefore forms a sparse dependency DAG in general; the recurrence theorem studies only the asymptotic foundational-core regime of monolithic, globally acting extensions.

The central observation is that these obligations are stratified by historical depth. Some are purely unary: the new layer must act on one previously exported generator. Others are genuinely binary: the new layer must preserve a previously exported compatibility witness between two generators. The main invariant of the paper measures the least depth at which this process stabilizes.

1.2. **The fixed theorem object.** This paper fixes one cubical object theory from the start. Let C be a CCHM-style cubical core calculus with univalence and Kan composition [CCHM18, CHM18]. Over C we study a language of globally acting extensions exported through opaque public signatures. A sealing step exposes only the normalized payload and trace fields of that public signature; the internal derivation that discharged the obligations is hidden behind opacity. The recurrence law proved below is therefore a theorem about a fixed cubical extension calculus built over C, not a blanket claim about every system that carries the label "cubical." Cubical Agda enters in two roles only: as a mechanization environment and as a source of native cubical constructions used to witness the lower-bound phenomena.

1.3. **The core thesis.** We call that invariant the *coherence depth* (or *coherence window*) of the foundation. The thesis of the paper is that, once coherence depth is isolated, the asymptotic law of integration becomes forced.

> In the fixed fully coupled cubical extension calculus of this paper, minimal opaque extension debt obeys a depth-two affine recurrence; under uniform payload size, the shifted sequence is Fibonacci.

Here "fully coupled" refers only to the foundational-core regime just described: admissible extensions are globally acting payloads whose semantics or promoted action is intended to range over the whole active interface, not orthogonal additions that leave most earlier exports untouched. Within that regime, the operative assumption is the scope of the advertised global action, not a prior density axiom. 2.21 identifies the primitive active *basis* interface of

this fixed extension calculus: native core extensions are constrained by canonicity-preserving totality of the operational semantics, while promoted interface packages are constrained by operational exhaustiveness of the exported global interface. The resulting recurrence is stated over the presentation-invariant minimal opaque cost μ, not over a chosen surface presentation. In extensional systems with UIP, the coherence depth collapses to 1, and under uniform payload size the recurrence is first-order affine. In the fixed cubical extension calculus studied here, the coherence depth is 2, and after the constant-payload shift the affine law becomes Fibonacci.

1.4. **Summary of contributions.** The paper develops one theorem package with three main components.

(1) We fix a cubical extension calculus over a chosen CCHM-style core, formalize its states, surface extensions, elaboration, sealed extensions, promoted interface packages, active interfaces, and integration latency, and distinguish orthogonal extensions from globally acting foundational core extensions. The payload and latency counts are defined on flattened telescope normal forms, so they are invariant under admissible interface refactorings such as rebundling records or splitting one interface into several shells. The bridge section then proves that elaboration, sealing, and counting normalization turn explicit surface declarations into canonical normalized public signatures, so the abstract counts are theorems about extension syntax over the fixed cubical core. Within this framework we prove a basis-sensitive coverage/minimality theorem for the fully coupled foundational-core regime, a payload-aware *Integration Trace Principle*, and a presentation-invariant minimal opaque cost μ measuring the irreducible trace fields that survive in canonical public form.

(2) We define coherence depth for this extension calculus and prove the sharp cubical/extensional dichotomy. In extensional type theory with UIP, the depth is 1. In the fixed fully coupled cubical extension calculus, the depth is 2. The depth-two statement is supported by four complementary ingredients: an exact upper bound on normalized obligation sets showing $\mathcal{O}^{(k)}(X) \simeq \mathcal{O}^{(2)}(X)$ for $k \geq 2$; a separate surface-syntactic corollary showing that primitive historical fields of arity at least 3 are eliminated from μ-minimal public signatures by cubical Kan operations; a recent-history factorization theorem showing that every surviving binary clause factors through the previous two normalized signatures; and lower bounds from an explicit swap-induced binary sealing obstruction, read categorically by adjunction coherence, together with a topological exact family supplied by clutching constructions for principal bundles over spheres.

(3) We combine the trace principle with the coherence-depth theorem to obtain a universal affine recurrence for minimal opaque cost. Under constant payload size c, the depth-two cubical case specializes to

$$\mu_{n+1} = \mu_n + \mu_{n-1} + 2c,$$

and the shifted sequence $U_n = \mu_n + 2c$ is Fibonacci. The associated structural inflation factor still converges to the Golden Ratio.

The Cubical Agda development described in section 6 formalizes the abstract candidate-and-obligation calculus, the cubical upper/lower metatheory, the recent-history factorization chain, and the abstract recurrence package underlying 4.8, 4.9, 4.11, 4.13, 4.14, 4.16, 4.17, 5.6, 5.1, 5.2, 5.3, and 5.5. The surface-syntactic readings of 2.21, 2.22, 4.10, 4.12, and the μ-based recurrence are then obtained in the paper by combining those formal

metatheorems with the bridge results of section 3. section 6 records the exact Agda theorem names and the trusted boundary.

1.5. **Related work.** The higher-dimensional background comes from the interpretation of intensional identity types as weak ω-groupoids, developed by Lumsdaine [Lum10] and van den Berg and Garner [vdBG11]. Several cubical calculi and implementations appear in the literature; the operational setting fixed in this paper is the CCHM-style cubical core of Cohen, Coquand, Huber, and Mörtberg [CCHM18], together with higher inductive types and Kan composition [CHM18]. Cubical Agda [VMA19] provides the mechanization environment and the native cubical programs used in the development, but the theorem target of the paper is the extension calculus defined below, not the full Agda surface language. For modal examples we use explicit modal type-theoretic syntax rather than informal endofunctor language: dependent right adjoints give a canonical Fitch-style dependent presentation [BCM+20], and cubical multimodal type theory shows how such modalities interact with cubical structure [AKGB24]. For comparison with mixed weak/strict equality foundations, see the two-level presentations of Altenkirch, Capriotti, and Kraus and of Annenkov, Capriotti, Kraus, and Sattler [ACK16, ACKS23].

The coherence-theoretic side is informed by Mac Lane's coherence theorem [Lan63]. We also use Stasheff's analysis of higher associativity [Sta63] and the well-foundedness approach to coherence of Kraus and von Raumer [KvR19]. Our contribution is to turn coherence from a qualitative structural property into a quantitative invariant of extension debt.

The topological family used for the lower bound is the classical clutching description of principal bundles over spheres; for standard bundle-theoretic background we refer to Husemoller [Hus94]. The univalent and cubical perspective sits in the general landscape of homotopy type theory and univalent foundations [Uni13].

2. A fixed cubical extension calculus

We fix throughout a cubical extension calculus built over a chosen CCHM-style core C with univalence and Kan composition. Its admissible extensions are globally acting payloads exported through opaque public signatures, and later layers may depend only on that public interface. The library therefore grows as a sequence

$$\mathcal{B}_0 \rightsquigarrow \mathcal{B}_1 \rightsquigarrow \mathcal{B}_2 \rightsquigarrow \cdots$$

of states, where each step adjoins one new sealed layer of the fixed extension calculus.

2.1. States, extensions, and promoted packages.

Definition 2.1 (State)**.** A *state* or *library* is a monotone context $\mathcal{B}$ closed under derivability. An evolution step

$$\mathcal{B}_n \rightsquigarrow \mathcal{B}_{n+1}$$

adjoins one sealed integration layer, denoted L_{n+1}.

Definition 2.2 (Surface extensions)**.** Over a state $\mathcal{B}_n$, a *surface extension declaration* is a finite quadruple

$$e = (\Gamma_{\mathrm{new}}, \Gamma_{\mathrm{act}}, \Gamma_{\mathrm{cmp}}, \sigma),$$

where Γ_{new} is a telescope of newly declared payload symbols, Γ_{act} is a finite family of unary action clauses against the active interface, Γ_{cmp} is a finite family of comparison or transport

clauses, and σ specifies which elaborated fields are exported through the sealed public interface. The admissible extensions studied in this paper are exactly those declarations whose intended action is global on the designated active interface footprint.

Definition 2.3 (Elaboration, sealing, and normalized public signature)**.** Each admissible surface extension declaration e over $\mathcal{B}_n$ elaborates to a core object

$$\mathsf{Elab}(e)$$

of the fixed cubical core C: the payload telescope elaborates to new core declarations, the action and comparison clauses elaborate to fields witnessing interaction with the active interface, and the opacity component σ selects the public telescope exported after sealing. The *normalized public signature* of e is the counting normal form

$$N(e) := N(\mathsf{Elab}(e)),$$

obtained by flattening the public telescope and discarding fields uniformly synthesized by definitional equality or the generic structural operations of the ambient cubical calculus.

Definition 2.4 (Presentation equivalence and minimal opaque cost)**.** Two admissible surface extensions e and e' over the same state are *presentation-equivalent*, written

$$e \approx e',$$

when their elaborated sealed extensions determine the same public typing judgment up to definitional equality, canonical telescope isomorphism, and transparent internal normalization. Write $N_{\mathrm{tr}}(e) \subseteq N(e)$ for the irreducible trace schemas in the normalized public signature of e. The *minimal opaque cost* of e is

$$\mu(e) := \min\{\, |N_{\mathrm{tr}}(e')| \mid e' \approx e \,\}.$$

Definition 2.5 (Candidate)**.** A *candidate* over a library state $\mathcal{B}$ is a pair

$$X = (X_{\mathrm{core}}, \mathcal{G}_{\mathrm{obl}}),$$

where X_{core} is the mathematical content proposed for addition and $\mathcal{G}_{\mathrm{obl}}$ is the family of normalized atomic coherence obligations induced when X is sealed against the active interface of $\mathcal{B}$. Every admissible surface extension declaration e determines such a candidate by elaboration.

Definition 2.6 (Counting normal form and atomic schemas)**.** Every public interface in this paper is counted only up to *canonical telescope isomorphism*: reassociation and flattening of iterated dependent sums, splitting or rebundling one single-constructor record into several nested records, and currying or uncurrying iterated function arguments when these preserve the same sealed typing judgment. Concretely, an exported interface is read as a dependent record, equivalently an iterated Σ-type whose field types may themselves begin with Π-telescopes. Its *counting normal form* is the fully flattened telescope obtained by reassociating the Σ-spine and expanding such record wrappers.

An *atomic schema* is one irreducible generating field of this counting normal form. A field is irreducible when its inhabitant is not uniformly synthesized from earlier fields by definitional equality, reflexivity, or the generic structural operations of the ambient calculus.

Definition 2.7 (Core payload size)**.** For a candidate X, let $P(X)$ be the finite set of atomic payload schemas appearing in the counting normal form of the public interface contributed

directly by X_{core} before any coherence witnesses against earlier layers are added. Its cardinality

$$\kappa(X) := |P(X)|$$

is the *core payload size* of X.

Definition 2.8 (Sealed extension)**.** A *sealed extension* is a candidate whose obligations have been discharged and whose resolved data are exported only through an opaque public interface. Future extensions may use those exports, but may not inspect the internal derivations that produced them.

Definition 2.9 (Promoted interface package)**.** A *promoted interface package* over $\mathcal{B}$ is a finite package of constructively irreducible generators and interaction data that is exported as one new sealed layer. Such a package need not modify the reduction rules of the kernel. What matters is that its public generators become part of the active exported interface for later sealing steps.

Definition 2.10 (Integration latency)**.** The *integration latency* of a candidate X over $\mathcal{B}$ is

$$\Delta(X \mid \mathcal{B}) := |\mathcal{G}_{\mathrm{obl}}|,$$

the number of irreducible atomic obligations that must be resolved before X can be exported as a sealed layer.

Remark 2.11 (Refactoring invariance of the counting notions)**.** Because κ, Δ, and later historical support are computed on counting normal forms, admissible refactorings such as bundling laws into one record, splitting one interface into several record shells, exporting curried rather than uncurried arguments, or pushing transparently derivable witnesses deeper into an internal term do not change the counts. Only a change in the irreducible primitive fields of the sealed typing judgment changes the metrics.

For a sealed layer L_k obtained from a candidate X_k, we write $S(L_k)$ for the set of atomic exported schemas in the counting normal form of its public interface and

$$\kappa_k := \kappa(X_k)$$

for its core payload size. The indexing is chosen so that Δ_k denotes the sealing cost of L_k.

Proposition 2.12 (Transparent user-level code lies outside the recurrence)**.** *Let U be a family of transparent definitions, lemmas, or record packages over $\mathcal{B}$ whose symbols are definitionally reducible using the existing operational semantics and are not promoted to a new opaque exported layer. Then U contributes zero integration latency and does not alter the active interface seen by later sealing steps.*

Proof. Such a development creates no new irreducible interaction site against the exported interface. All reductions remain inherited from the pre-existing library, and no new opaque generators are added to the global interface telescope. Hence U is ordinary growth inside one state, not a state transition of the form $\mathcal{B}_n \rightsquigarrow \mathcal{B}_{n+1}$. □

Definition 2.13 (Foundational core extension)**.** A candidate X over $\mathcal{B}_n$ is a *foundational core extension* if its payload is intended to act globally on the currently active interface: each atomic payload generator in $P(X)$ is operationally or semantically incomplete until its behavior has been specified against every generator in the designated coupling footprint inside the active exported interface. Typical examples are extensions of a universe-decoding

apparatus, global modalities presented by explicit modal dependent-type syntax whose context action must distribute across existing type formers and transports [BCM+20, AKGB24], and heavily promoted generic interfaces whose exported action is library-wide.

Remark 2.14. Not every sealed extension is foundational in the sense of 2.13. A new datatype or API layer may be orthogonal to much of the existing library and therefore require only sparse interaction data. Such extensions may preserve canonicity without becoming globally coupled to earlier layers. The recurrence theorem of this paper is intentionally a worst-case statement about foundational core extensions, not a universality claim about all sealed library growth.

This scope restriction is essential for the basis-coverage theorem below. Ordinary user-level additions typically enlarge a sparse DAG of modules or datatypes and do not force primitive interaction with unrelated earlier exports. The maximal-coupling statement proved here isolates the special foundational-core regime in which the new sealed layer is advertised as a *global* operation on the active interface, so that canonicity and global closure become exhaustiveness requirements rather than bookkeeping conventions.

2.2. **Global action, basis coverage, and canonical minimality.** To speak about coupling to historical layers, it is convenient to parameterize the visible interface by depth.

Definition 2.15 (Historical interface)**.** For $k \geq 1$, the *historical interface of depth* k at stage n is

$$I_n^{(k)} := \biguplus_{j=0}^{k-1} S(L_{n-j}),$$

the tagged coproduct of the atomic schemas exported by the most recent k sealed layers in counting normal form.

Definition 2.16 (Chosen basis of the historical interface)**.** For each sealed layer L, fix a finite family

$$S_{\mathrm{bas}}(L) \subseteq S(L)$$

of *basis schemas* with the following properties:

(1) every field of $S(L)$ is uniformly generated from $S_{\mathrm{bas}}(L)$ by definitional equality, uniform polymorphic instantiation, transport along already exported equalities, or the generic structural operations already quotiented in 2.6;
(2) no proper subfamily of $S_{\mathrm{bas}}(L)$ still has property (1).

For $k \geq 1$, the *historical basis of depth* k at stage n is

$$I_{n,\mathrm{bas}}^{(k)} := \biguplus_{j=0}^{k-1} S_{\mathrm{bas}}(L_{n-j}).$$

Its elements are the *basis sites* on which a globally acting extension is primitively specified. Different valid choices of basis are canonically bijective by the same counting-normalization and refactoring invariance principles used throughout the paper, so only the resulting cardinality matters.

Definition 2.17 (Basis coverage and canonical basis minimality)**.** An admissible foundational core candidate sealed against $I_n^{(k)}$ has *basis coverage* if every basis site of $I_{n,\mathrm{bas}}^{(k)}$ receives at least one primitive interaction datum from the new extension. It is in *canonical basis-minimal form* if, after elaboration and normalization, each basis site contributes at most one irreducible public trace schema. Equivalently, presentation-equivalent duplicate clauses have been eliminated from the normalized public signature.

Definition 2.18 (Fully coupled foundation)**.** A foundation is *fully coupled* if the admissible sealed extensions under study are foundational core extensions whose public signatures are advertised as acting on the whole active historical interface. Equivalently, the paper's admissible extensions in a fully coupled foundation are globally acting payloads whose intended coupling footprint is the whole active interface, not orthogonal additions with sparse dependency patterns. This definition fixes only the scope of the action. The coverage/minimality theorems below derive the exact irreducible primitive interaction data required to realize that action.

The next theorem turns basis coverage into a metatheoretic necessity for this globally coupled regime rather than a stylistic choice.

Theorem 2.19 (Global admissibility forces basis coverage)**.** *Let $\mathcal{B}_n$ be a library state in a fully coupled foundation, let X be an admissible foundational core extension sealed against the historical interface $I_n^{(k)}$, and let $I_{n,\mathrm{bas}}^{(k)}$ be a chosen basis for that interface. If the extended theory is to pass the sealed library's global admissibility discipline, then every basis site of $I_{n,\mathrm{bas}}^{(k)}$ must receive at least one primitive interaction datum from X. For native core extensions this is enforced by canonicity-preserving totality of the operational semantics, while for promoted interface packages it is enforced by operational exhaustiveness of the exported global interface.*

Proof. By 2.18, the admissible candidates considered here are not arbitrary sealed additions but foundational core extensions whose payload is intended to act on the whole active interface. The theorem is therefore about that global regime only; orthogonal extensions of the kind noted in 2.14 are outside its hypotheses.

First consider a native global computational extension. In a Tarski-style universe presentation, suppose the active interface exports a code universe U and a decoding map $\mathsf{El} : U \to \mathsf{Type}$. If the new sealed layer extends the universe with a fresh code but the extended decoding machinery omits one basis site of the active interface, then by definition of 2.16 there remains a family of raw exported generators whose behavior is not recovered from the remaining primitive clauses by definitional equality, polymorphic instantiation, transport, or generic structural operations. Closed terms mentioning that uncovered family no longer reduce through El, so canonicity fails on the extended language. The same phenomenon occurs for any native global operator advertised as acting across the active interface: omitting one irreducible basis site leaves a well-typed class of terms at a stuck neutral or partially defined reduction site. Thus native admissibility forces at least one clause for every basis site of $I_{n,\mathrm{bas}}^{(k)}$.

Next consider a promoted interface package. Such a package need not alter kernel reduction, so standard canonicity is not the right notion. What matters instead is operational exhaustiveness of the exported global interface. If the package is advertised as a library-wide action, its public typing judgment is a total dependent map out of the record enumerating

the active interface. Omitting one basis site again leaves an irreducible family of cases uncovered, because that site's action is not derivable from the other clauses by the transparent operations built into 2.16. Later sealing steps cannot therefore treat the package as a closed global operation. Thus promoted packages also require at least one primitive witness for every basis site of $I^{(k)}_{n,\mathrm{bas}}$. □

Proposition 2.20 (Canonical normalized presentations are basis-minimal)**.** *Let e be an admissible surface extension declaration over $\mathcal{B}_n$ whose candidate $X(e)$ is sealed against the historical interface $I^{(k)}_n$. Then the normalized public signature $N(e)$ contains at most one irreducible public trace schema for each basis site of $I^{(k)}_{n,\mathrm{bas}}$. Equivalently, redundant primitive clauses that differ only by transparent polymorphic compression, definitional equality, transport, or generic structural operations are eliminable from the canonical normalized presentation.*

Proof. Suppose two primitive clauses of e correspond to the same basis site. By construction of 2.16, they induce the same public trace field up to definitional equality, transparent transport, uniform polymorphic instantiation, or the generic structural operations already quotiented by counting normalization. If both survived as distinct irreducible fields in $N(e)$, then the sealed public signature would contain presentation-equivalent duplicate trace data. But 3.2 says that $N(e)$ retains exactly the irreducible visible public fields after normalization, and 3.4 identifies a canonical normalized presentation for each presentation-equivalence class. Hence one of the two clauses is redundant and disappears in canonical normalized form. Therefore at most one irreducible trace schema survives per basis site. □

Theorem 2.21 (Basis-sensitive density law)**.** *Let $\mathcal{B}_n$ be a library state in a fully coupled foundation, let X be an admissible foundational core extension sealed against the historical interface $I^{(k)}_n$, and let $I^{(k)}_{n,\mathrm{bas}}$ be a chosen basis for that interface. Then*

$$\Delta(X \mid \mathcal{B}_n) = |I^{(k)}_{n,\mathrm{bas}}|.$$

In particular, sealing cost counts irreducible basis clauses rather than the raw number of exported generators.

Proof. Coverage is exactly 2.19. Minimality is exactly 2.20 after passing from surface declarations to their canonical normalized public signatures. Since $\Delta(X \mid \mathcal{B}_n)$ counts irreducible primitive obligations in that normalized presentation, every basis site contributes one and only one irreducible primitive trace field. Therefore

$$\Delta(X \mid \mathcal{B}_n) = |I^{(k)}_{n,\mathrm{bas}}|.$$

□

2.3. **The integration trace principle.** In the fully coupled foundational-core regime, the preceding theorem identifies sealing cost with the size of the active basis interface. The next theorem explains how those resolved obligations reappear in the next layer's public export alongside the layer's own new mathematical payload.

Theorem 2.22 (Integration Trace Principle)**.** *Each sealed integration layer exports, in canonical basis form, a disjoint union of two kinds of public schema: the new payload contributed by its core content and the trace schemas corresponding to its resolved basis obligations. Equivalently, if L_k is the layer obtained by sealing a candidate of core payload size κ_k and coherence cost Δ_k, then there is a canonical decomposition*

$$S_{\mathrm{bas}}(L_k) \cong S_{\mathrm{core}}(L_k) \uplus S_{\mathrm{tr}}(L_k)$$

with

$$|S_{\mathrm{core}}(L_k)| = \kappa_k, \qquad |S_{\mathrm{tr}}(L_k)| = \Delta_k.$$

Hence

$$|S_{\mathrm{bas}}(L_k)| = \kappa_k + \Delta_k.$$

Proof. There are two logically distinct sources of basis schemas in the canonical normalized presentation.

First, the candidate's core content $X_{k,\mathrm{core}}$ contributes genuinely new public generators: a new type former, modality, promoted interface operator, or other primitive schema that did not previously exist in the library. By definition of core payload size, the number of these newly exported payload schemas is κ_k. These form $S_{\mathrm{core}}(L_k)$.

Second, by 2.21, sealing against the active interface requires exactly one irreducible primitive interaction datum for each basis site of the active interface. Formally, a sealed layer is exported as a dependent record, equivalently an iterated Σ-type, whose fields consist of the new payload together with the coherence cells needed to type that payload against earlier exports. Opacity hides the inhabitants of those fields, but not the field-types occurring in the exported signature. Thus once such a datum is supplied and the layer is sealed, it no longer survives as an inspectable derivation visible to future extensions; it survives as a public trace field recording how the new payload acts on that older basis site. These schemas form $S_{\mathrm{tr}}(L_k)$, and there is one of them for each resolved basis obligation. Hence $|S_{\mathrm{tr}}(L_k)| = \Delta_k$.

Crucially, this decomposition is exact at the level of the sealed typing judgment. Replacing a primitive witness by a deeper internal derivation can change the hidden inhabitant of a trace field, but it cannot remove the corresponding field-type from the exported telescope without changing the public interface against which later layers are checked.

Because the layer is sealed, every basis schema of L_k is of one of these two kinds: either it is a newly introduced payload symbol, or it is the exported trace of a resolved obligation attaching that payload to prior interface data. The two kinds are disjoint, since payload symbols are primitive generators introduced by the new layer itself, whereas trace schemas are indexed by the prior basis interface against which the new layer was sealed. Therefore the canonical basis export decomposes as

$$S_{\mathrm{bas}}(L_k) \cong S_{\mathrm{core}}(L_k) \uplus S_{\mathrm{tr}}(L_k),$$

and the cardinality formula follows immediately. □

Remark 2.23. The trace principle counts kernel-level extensions and promoted interface packages uniformly. In both cases the public export has a payload part and a trace part. The only difference is the kind of primitive datum involved: for native extensions the payload contributes new reduction heads and the trace contributes defining clauses, whereas for promoted packages the payload contributes new exported operators and the trace contributes comparison or transport witnesses.

3. From cubical syntax to normalized extension signatures

The counting calculus of section 2 is intended to measure explicit extension declarations over the fixed cubical core C, not a freestanding payload calculus detached from syntax. This section spells out that bridge. Surface extensions elaborate to sealed core objects, sealing exports only their opaque public interfaces, and counting is performed on the normalized public signature of that export.

3.1. **Elaboration and public signatures.**

Theorem 3.1 (Elaboration soundness for sealed exports)**.** *Every admissible surface extension declaration e over a state $\mathcal{B}_n$ determines:*

(1) *an elaborated candidate*
$$X(e) := (X_{\mathrm{core}}(e), \mathcal{G}_{\mathrm{obl}}(e)),$$
(2) *a sealed layer $L(e)$ exported through the opaque public interface selected by σ,*
(3) *and a normalized public signature*
$$N(e) = N(\mathsf{Elab}(e))$$
obtained from that exported interface by counting normalization.

In particular, the candidate calculus of section 2 is the image of admissible surface extensions under elaboration and sealing.

Proof. By 2.2 and 2.3, the payload telescope Γ_{new} elaborates to new core declarations, while Γ_{act} and Γ_{cmp} elaborate to the clauses witnessing the required interaction with the active interface of $\mathcal{B}_n$. Admissibility means precisely that these clauses type-check and discharge the induced coherence obligations, so the result packages into a candidate $X(e)$ together with an opaque exported layer $L(e)$. Applying the counting normal form of 2.6 to the resulting public telescope yields $N(e)$. Thus every admissible surface extension gives rise canonically to the candidate-and-sealed-layer data used in section 2. □

Theorem 3.2 (Adequacy of normalized public signatures)**.** *Let e be an admissible surface extension over $\mathcal{B}_n$ and let $L(e)$ be its sealed export. Then:*

(1) *every field of $N(e)$ comes from a field of the sealed public interface of $L(e)$; counting normalization introduces no new primitive public datum;*
(2) *every field of $N(e)$ is either a payload schema contributed by Γ_{new} or a trace schema arising from a resolved obligation of $X(e)$;*
(3) *every irreducible payload or trace schema visible after sealing occurs as a field of $N(e)$;*
(4) *the synthetic quantities $P(X(e))$, $S(L(e))$, $\Delta(X(e) \mid \mathcal{B}_n)$, $\mu(e)$, and $\mathcal{O}^{(k)}(X(e))$ are exactly the corresponding payload, public-signature, abstract latency, minimal opaque cost, and obligation sets computed from $N(e)$ and its trace part.*

Proof. Point (1) is immediate from 2.3 and 2.6: the normalization procedure only flattens the exported telescope and discards uniformly synthesized fields, so it cannot create a new primitive field not already present in the sealed typing judgment. Point (2) is the surface-language form of the decomposition proved abstractly in 2.22: after elaboration and sealing, every visible field is either newly introduced payload or the public trace of a resolved interaction with prior interface data. Point (3) follows because counting normalization removes only presentation artifacts: reassociation of telescopes, transparently derivable fields, and other data uniformly synthesized by definitional equality or the generic structure of

the ambient cubical calculus. Thus every irreducible field that remains visible after sealing survives into $N(e)$, and no extra irreducible field is created by normalization. Point (4) is then immediate from the definitions of payload size, public schema set, minimal opaque cost, integration latency, and induced obligation sets in section 2 and section 4: those quantities are computed exactly from the normalized public signature and its induced obligations. □

Proposition 3.3 (Necessity of irreducible public fields)**.** *Let e be an admissible surface extension over $\mathcal{B}_n$, and let ϕ be an irreducible field of $N(e)$. If $e' \approx e$, then $N(e')$ contains a canonically corresponding irreducible field ϕ'. Equivalently, no presentation-equivalent declaration can delete an irreducible payload or trace schema from the sealed public signature.*

Proof. By presentation equivalence, e and e' determine the same sealed public typing judgment up to definitional equality, canonical telescope isomorphism, and transparent internal normalization. If the irreducible field ϕ were absent from $N(e')$, then one of two things would have to happen: either the public typing judgment of e' would genuinely omit that field, contradicting $e' \approx e$, or the field would have to be uniformly reconstructible from the remaining exported data by those transparent operations. But 2.6 declares ϕ irreducible precisely by excluding that second possibility. Hence ϕ must survive in every presentation-equivalent normalized public signature. □

Proposition 3.4 (Canonical normalized presentations)**.** *If $e \approx e'$, then $N(e)$ and $N(e')$ are canonically isomorphic, and their trace parts $N_{\mathrm{tr}}(e)$ and $N_{\mathrm{tr}}(e')$ are canonically isomorphic as well. Consequently every presentation-equivalence class admits a canonical normalized public signature up to isomorphism, and*

$$\mu(e) = |N_{\mathrm{tr}}(e)|.$$

Proof. Presentation equivalence identifies elaborated sealed extensions whenever they induce the same public typing judgment up to definitional equality, canonical telescope isomorphism, and transparent internal normalization. The counting normal form forgets exactly those presentation differences and nothing more, so $N(e)$ and $N(e')$ are canonically isomorphic. By 3.2, the payload/trace decomposition is part of the sealed typing judgment itself, so the trace subfamilies correspond under that isomorphism. By 3.3, no presentation-equivalent declaration can remove an irreducible trace field from the normalized public signature unless that field was already transparently reconstructible. Hence every representative of the class has the same canonical irreducible trace cardinality, so the minimum in 2.4 is attained by $N_{\mathrm{tr}}(e)$ itself and equals $|N_{\mathrm{tr}}(e)|$. □

Corollary 3.5 (Minimal opaque cost agrees with abstract coherence cost)**.** *For every admissible surface extension declaration e over $\mathcal{B}_n$,*

$$\mu(e) = \Delta(X(e) \mid \mathcal{B}_n).$$

If e_k is the declaration whose sealing exports L_k, and if $\mu_k := \mu(e_k)$, then

$$|S_{\mathrm{bas}}(L_k)| = \kappa_k + \mu_k.$$

Proof. By 3.2, the trace part of $N(e)$ records exactly the irreducible public trace schemas arising from resolved obligations of $X(e)$. By 3.4, $\mu(e)$ is exactly the cardinality of that canonical trace part. By 2.21, there is one and only one irreducible primitive trace schema for each basis site of the active interface, so that cardinality is precisely $\Delta(X(e) \mid \mathcal{B}_n)$. The final formula is then the trace-principle decomposition of 2.22, rewritten using $\mu_k = \Delta(X(e_k) \mid \mathcal{B}_{k-1})$. □

Remark 3.6 (Syntactic scope of the bridge)**.** The results of 3.1, 3.2, 3.3, and 3.4 are syntactic bridge theorems: they speak only about elaboration, sealing, opacity, counting normalization, and the presentation-equivalence relation of 2.4. They justify that $N(e)$ and $\mu(e)$ are invariants of explicit extension declarations over the fixed cubical core C.

By contrast, 2.21, 2.22, and the later depth and recurrence theorems are metatheorems of the abstract candidate-and-obligation calculus. Their application to surface syntax passes through the adequacy theorem above: once an admissible declaration has been elaborated and normalized, those later counts are read off from the resulting normalized public signature. Throughout the rest of the paper we keep the definitions of $N(e)$ and $\mu(e)$ from 2.3 and 2.4 unchanged; all later minimality statements are therefore relative to this fixed presentation-equivalence relation.

4. The coherence window concept and the depth dichotomy

4.1. Induced obligations and historical arity.

Definition 4.1 (Induced obligations)**.** Let $\mathcal{O}^{(k)}(X)$ be the set of normalized atomic obligation types induced when candidate X is sealed against a historical interface of depth k. Each element is read in the counting normal form of 2.6: we quotient by the same canonical telescope isomorphisms and retain only irreducible generating fields not uniformly synthesized by the ambient calculus. Here normalization retains exactly the developer-visible data not already exported by prior layers. In particular, when a potential higher-dimensional coherence problem appears with one face left free, the corresponding atomic obligation is represented by the horn-extension type of the missing face together with a filler, rather than by the type of fillers for a completely fixed boundary. Since the interface grows cumulatively, there are inclusions

$$\mathcal{O}^{(1)}(X) \subseteq \mathcal{O}^{(2)}(X) \subseteq \mathcal{O}^{(3)}(X) \subseteq \cdots .$$

Remark 4.2. Three notions must be kept distinct. An obligation may be absent; it may be present but contractible; or it may be present while still being eliminable from every μ-minimal public presentation. 4.6 is a statement about exact normalized obligation types. The surface-syntactic quantity μ asks a different question, namely which trace fields survive in canonical normalized public form. Accordingly, "derivable", "contractible", "eliminable from a minimal presentation", and "not present" are never used interchangeably in this paper.

Definition 4.3 (Historical support and arity)**.** For a normalized obligation o, let $\mathrm{Supp}(o)$ be the set of sealed layers whose atomic exported schemas occur irreducibly in the counting normal form of the typing derivation of o after each prior layer has been collapsed to its opaque interface. Its *historical arity* is

$$a(o) := |\mathrm{Supp}(o)|.$$

Remark 4.4. Historical arity counts independent prior layers, not textual age. A direct lookup to an old exported theorem is still unary, because it lands in one opaque interface. Multi-layer debt arises only when the new layer must cohere simultaneously with several independent prior exports.

Lemma 4.5 (Arity-to-dimension correspondence)**.** *An irreducible obligation of historical arity k requires a k-dimensional coherence cell. Arity 1 produces substitution or transport data, arity 2 produces a naturality square or homotopy, and arity $k \geq 3$ would require a genuine higher filler.*

Proof. With one historical input, the new layer must supply a comparison morphism between its behavior and one prior export, so the obligation is one-dimensional. With two independent historical inputs, the witness compares two unary composites and therefore has the form of a square or homotopy. More generally, relating k independently supplied historical interfaces requires a k-dimensional filler between the corresponding composites. □

4.2. **Definition of coherence depth.**

Definition 4.6 (Coherence depth)**.** A foundation, equivalently a fixed extension calculus together with its sealing discipline, has *coherence depth* d if d is the least integer such that for every candidate X and all $k \geq d$ there is a canonical equivalence

$$\mathcal{O}^{(k)}(X) \simeq \mathcal{O}^{(d)}(X).$$

We also call d the *coherence window* of the foundation.

The point of 4.6 is that beyond depth d, the normalized obligation types themselves stop changing up to canonical equivalence. This is stronger than saying merely that deeper history contributes no new minimal opaque cost. Once that exact stabilization has been upgraded to a genuine chronological window, the size of the primitive active basis interface controls the recurrence law for integration debt.

Remark 4.7. 4.6 is a stabilization statement about *obligation sets*. It should not be read as saying that a bound on coherence dimension automatically identifies the chronological layers that must still be inspected primitively. In particular, an arity-two obligation could in principle couple a new layer to a remote historical layer. To extract a genuine time window from a dimensional bound one needs an additional factorization theorem showing that such remote binary comparisons are already generated by recently exported traces. In the cubical univalent case, this passage from arity to chronology is supplied by 4.12.

4.3. **Theorem A: extensional systems collapse to $d = 1$.**

Theorem 4.8. *In Martin-Löf type theory with UIP, the coherence depth is* 1*.*

Proof. Under UIP, any two proofs of identity between fixed endpoints are equal. Hence every identity type is a set, and every higher identity type is a proposition; once inhabited, it is contractible. Consider any normalized coherence obligation of dimension at least 2. By 4.5, it corresponds to a higher identity witness. But in the presence of UIP such a witness is uniquely determined by its boundary and therefore carries no independent data. Thus every obligation of arity at least 2 collapses to data already fixed by its unary boundary. Equivalently,

$$\mathcal{O}^{(k)}(X) \simeq \mathcal{O}^{(1)}(X) \qquad (k \geq 1).$$

Since depth 0 would mean the absence of any interaction with the current interface, which is impossible for a nontrivial sealing step, the least stabilizing depth is 1. □

4.4. **Theorem B: fully coupled univalent systems have $d = 2$.** From now on we specialize the paper's fixed theorem object to the fully coupled cubical extension calculus built over the chosen CCHM-style core [CCHM18, CHM18], with univalence and Kan composition available in the ambient operational semantics. The next ingredients are deliberately complementary: exact obligation-set stabilization gives the upper bound on coherence depth, higher-coherence elimination gives the separate syntactic upper bound on μ-minimal public presentations, recent-history factorization turns that depth bound into a genuine two-step time window, and the explicit swap obstruction together with the clutching family give lower-bound witnesses. Together they pin down exact depth 2.

One semantic clarification is important before the cubical argument begins. In univalent foundations, types behave as weak ω-groupoids, so the geometric space of fillers for a fixed higher boundary is not generally contractible. The paper therefore does not model a depth-three obligation as the space of fillers for a closed 3-sphere. After the normalization of 4.1, the marginal higher-dimensional datum appears instead as a horn-extension problem: an open box with one remote face still free.

It is equally important to delimit the class of obligations under discussion. The paper does not claim that arbitrary internal higher-algebraic structure carried by a candidate disappears from cubical type theory. What is measured here is only *structural integration coherence*: the obligations induced by sealing a globally acting foundational candidate against prior opaque layers. Those obligations arise from the chronological dependency graph of exported payload and trace data, not from arbitrary user-specified higher operations internal to the candidate.

Concretely, once the visible unary and binary traces have been fixed, the depth-three fiber has the form

$$H(\partial) := \sum_{\gamma:\Gamma_{\mathrm{miss}}(\partial)} \mathsf{Fill}_{\partial}(\gamma),$$

where ∂ is the already exported boundary, γ is the missing remote binary face, and $\mathsf{Fill}_{\partial}(\gamma)$ is the type of higher fillers completing the open box once γ has been chosen. This is a horn-extension type, not a closed-filler type. In cubical type theory, the operational semantics provide generic Kan operations such as hcomp, comp, and transp that solve such horn-extension problems uniformly. The sharp question for this paper is therefore syntactic: does a field of that shape survive as an irreducible trace field in a μ-minimal normalized public signature, or can it be replaced definitionally by a term synthesized from lower-arity boundary data?

Lemma 4.9 (Syntactic horn reduction for structural integration coherence)**.** *Let X be a foundational core candidate sealed at stage $n+1$ in the fixed fully coupled cubical extension calculus. For every $k \geq 3$ and every boundary $\partial : \mathcal{O}^{(k-1)}(X)$, the additional data exposed by adjoining the remote layer L_{n-k+1} are canonically represented by a horn-extension fiber*

$$H_k(\partial) := \sum_{\gamma:\Gamma^{(k)}_{\mathrm{miss}}(\partial)} \mathsf{Fill}_{\partial}(\gamma).$$

Here γ is the new remote comparison face between X and L_{n-k+1}, while every other face of the box is already exported by the unary and binary traces of the intervening sealed layers.

Proof. By construction, $\mathcal{O}^{(k)}(X)$ records only structural integration obligations induced by sealing X against opaque prior layers. Earlier layers therefore contribute only their exported payload and trace schemas; arbitrary higher-algebraic structure internal to those layers

is invisible at sealing time. Passing from depth $k-1$ to depth k exposes exactly one additional remote layer, namely L_{n-k+1}. By 2.21, the global action of X contributes exactly one interaction site for each newly exposed generator. Meanwhile, because the intermediate layers were already sealed, their exports already contain the unary bridges and binary comparison traces linking that remote layer to the more recent history. After normalization, all visible faces of the resulting comparison cube are therefore fixed except the direct face comparing the action of X with the newly exposed remote layer. The remaining datum is precisely that missing face together with the higher filler certifying compatibility with the already exported boundary. This is exactly the displayed horn-extension fiber. □

A concrete cubical sketch. The abstract distinction between payload and trace can be made visible in a small cubical example. Suppose an older sealed layer exports a small Tarski universe with function codes and their decoding equation. A new cohesive layer contributes a genuinely new modal payload `Flat` together with its unit `eta`. Everything else needed to make that modality act on the older universe is integration trace: a lifted action on old codes, a comparison between decoding before or after lifting, and the distribution law over old function codes. In the snippet below, the witness `remote-face` is the point of the example. It is not additional payload. It is exactly the missing comparison face of the open 3-box determined by the old decoding law together with the new unary and binary trace data, and cubical composition computes it uniformly.

```
-- Schematic Cubical Agda: old universe + new payload + induced trace

record OldU : Set1 where
  field
    U      : Set
    El     : U -> Set
    Arr    : U -> U -> U
    El-Arr : (A B : U) ->
      Path Set (El (Arr A B)) (El A -> El B)

record Payload : Set1 where
  field
    Flat : Set -> Set
    eta  : {A : Set} -> A -> Flat A

record Trace (L : OldU) (P : Payload) : Set1 where
  open OldU L
  open Payload P
  field
    FlatU         : U -> U
    flat-El       : (A : U) ->
      Path Set (El (FlatU A)) (Flat (El A))
    flat-code-Arr : (A B : U) ->
      Path U (FlatU (Arr A B)) (Arr (FlatU A) (FlatU B))
    flat-Arr      : (A B : U) ->
      Path Set (Flat (El A -> El B))
```

```
                (Flat (El A) -> Flat (El B))

module _ (L : OldU) (P : Payload) (T : Trace L P) where
  open OldU L
  open Payload P
  open Trace T

  -- left route:
  --   El (FlatU (Arr A B))
  --     -> Flat (El (Arr A B))
  --     -> Flat (El A -> El B)
  --     -> Flat (El A) -> Flat (El B)

  -- right route:
  --   El (FlatU (Arr A B))
  --     -> El (Arr (FlatU A) (FlatU B))
  --     -> El (FlatU A) -> El (FlatU B)
  --     -> Flat (El A) -> Flat (El B)

  postulate
    left-route  :
      (A B : U) ->
      Path Set (El (FlatU (Arr A B))) (Flat (El A) -> Flat (El B))
    right-route :
      (A B : U) ->
      Path Set (El (FlatU (Arr A B))) (Flat (El A) -> Flat (El B))

  -- The open 3-box says these two boundary composites agree.
  -- This face is derived trace, not new payload.
  remote-face :
    (A B : U) ->
    Path (Path Set (El (FlatU (Arr A B)))
                   (Flat (El A) -> Flat (El B)))
         (left-route A B) (right-route A B)
  remote-face A B = hcomp ...
```

Read top-to-bottom, `Payload` is the human-authored new content, while `Trace` is the compatibility debt forced by sealing against the old universe. The two routes are the two composites from `El (FlatU (Arr A B))` to `Flat (El A) -> Flat (El B)`. The marginal depth-3 obligation is not the space of fillers for a closed boundary. It is the horn-extension type whose free component is the remote comparison between those composites and whose second component is the 3-cell certifying compatibility with the displayed boundary. In a cubical presentation this total extension type is canonically trivial: `hcomp` computes the missing face and `hfill` packages the accompanying filler. The extra factor is therefore present but contractible, rather than absent.

Theorem 4.10 (Higher-coherence elimination from minimal public signatures)**.** *Let e be an admissible surface extension declaration over $\mathcal{B}_n$. Then there exists a presentation-equivalent*

declaration

$$e' \approx e$$

whose normalized trace signature $N_{\mathrm{tr}}(e')$ contains no irreducible historical field of arity $k \geq 3$. Equivalently, every μ-minimal normalized public signature contains only unary and binary primitive historical trace fields. More concretely, whenever a would-be primitive field of arity $k \geq 3$ appears in some presentation, it can be removed and replaced by a term synthesized from its lower-arity boundary data using cubical Kan operations, specifically hcomp*,* transp*, and the accompanying filler operation* hfill*.*

Proof. Let ϕ be a historical trace field of arity $k \geq 3$ appearing in $N_{\mathrm{tr}}(e)$. By 3.2, ϕ corresponds to a visible resolved obligation in the public signature of e. By 4.9, the additional datum contributed by that arity-k step is a horn-extension fiber $H_k(\partial)$ over a boundary ∂ already exported by lower-arity trace fields. Cubical Kan structure computes a canonical inhabitant of that fiber from the boundary alone: hcomp synthesizes the missing remote face, transp aligns the comparison along the relevant boundary transport, and hfill packages the resulting higher filler.

Replace the primitive witness for ϕ by that synthesized term internally before sealing. The resulting declaration is presentation-equivalent to e, because presentation equivalence already identifies elaborated declarations that differ only by definitional equality, canonical telescope isomorphism, or transparent internal normalization. The public typing judgment is unchanged, but the corresponding trace field is no longer irreducible: it is definitionally reconstructed from lower-arity boundary data and therefore disappears from the canonical normalized trace signature. Since the public trace signature is finite, iterating this elimination over all fields of historical arity at least 3 yields a presentation-equivalent declaration e' with no such irreducible fields. By 3.4, $\mu(e)$ is computed exactly on that canonical trace signature, so no field of arity at least 3 contributes to minimal opaque cost. □

4.4.1. *Dimensional upper bound ($d \leq 2$).*

Theorem 4.11 (Dimensional upper bound)**.** *In the fixed fully coupled cubical extension calculus, for every foundational core candidate X and every $k \geq 2$ there is a canonical equivalence*

$$\mathcal{O}^{(k)}(X) \simeq \mathcal{O}^{(2)}(X).$$

Hence the coherence depth is at most 2*.*

Proof. The case $k = 2$ is tautological. Let $k = 3$. By 4.5 and 4.9, the marginal higher-dimensional datum is a 3-cell presented pointwise over a depth-two boundary $\partial : \mathcal{O}^{(2)}(X)$ as the horn-extension type

$$H_3(\partial) := \sum_{\gamma:\Gamma_{\mathrm{miss}}(\partial)} \mathsf{Fill}_\partial(\gamma),$$

where γ is the missing remote binary face and $\mathsf{Fill}_\partial(\gamma)$ is the type of 3-cells completing the open box once γ has been chosen.

In cubical type theory, Kan composition solves such horn-extension problems uniformly. The operation hcomp computes a canonical missing face from the visible boundary data, and hfill supplies the corresponding filler [CCHM18, CHM18]. Therefore each fiber $H_3(\partial)$

is contractible. Equivalently, $\mathcal{O}^{(3)}(X)$ is a dependent sum over $\mathcal{O}^{(2)}(X)$ with contractible fibers, so

$$\mathcal{O}^{(3)}(X) \simeq \mathcal{O}^{(2)}(X).$$

Now let $k > 3$. By 4.9, passing from depth $k-1$ to depth k again adjoins only a horn-extension fiber $H_k(\partial)$ over each $\partial : \mathcal{O}^{(k-1)}(X)$: one more remote comparison face together with a filler against boundary data already exported by lower layers. The same cubical Kan operations contract that fiber. Hence

$$\mathcal{O}^{(k)}(X) \simeq \mathcal{O}^{(k-1)}(X)$$

for every $k \geq 3$, and iterating yields $\mathcal{O}^{(k)}(X) \simeq \mathcal{O}^{(2)}(X)$ for all $k \geq 2$. Separately, 4.10 gives the surface-syntactic corollary that any arity-$k \geq 3$ trace witness is definitional shadow of lower-arity boundary data and therefore disappears from a μ-minimal public presentation. □

Theorem 4.12 (Recent-history factorization)**.** *In the fixed fully coupled cubical extension calculus, every binary trace field that survives in a μ-minimal normalized public signature at stage $n+1$ factors through the previous two normalized signatures. More precisely, let X be a candidate added at stage $n+1$. Once the unary and binary trace data of X against L_n and L_{n-1} have been fixed, every comparison between X and a remote layer L_{n-r} with $r \geq 2$ is definitionally equal to a term obtained by iterating the factorization through the already exported unary bridges and binary comparison traces of the intervening normalized signatures. Consequently no μ-minimal public signature contains a primitive remote binary field.*

Proof. Start with the first nonadjacent layer L_{n-2}. Because L_n was itself sealed against L_{n-1} and L_{n-2}, its exported trace includes a unary comparison

$$f_n : L_n \to L_{n-1},$$

the previous layer exports

$$g_{n-1} : L_{n-1} \to L_{n-2},$$

and the binary part of the trace of L_n contains a 2-cell

$$\alpha_n : g_{n-1} \circ f_n \Rightarrow h_n$$

recording the induced comparison $h_n : L_n \to L_{n-2}$. When X is sealed against the last two layers, it supplies unary comparisons

$$x_n : X \to L_n, \qquad x_{n-1} : X \to L_{n-1},$$

and a binary coherence square

$$\beta_n : f_n \circ x_n \Rightarrow x_{n-1}.$$

The cells x_n, x_{n-1}, f_n, g_{n-1}, α_n, and β_n, together with the evident degenerate composition faces, determine an open 3-box. The marginal obligation is not merely a bare remote face; it is the horn-extension type

$$H_{n-2} := \sum_{\gamma_{n-2} : h_n \circ x_n \Rightarrow g_{n-1} \circ x_{n-1}} \mathsf{Fill}(\gamma_{n-2}),$$

whose first component is precisely the comparison

$$\gamma_{n-2} : h_n \circ x_n \Rightarrow g_{n-1} \circ x_{n-1}$$

between the two composites from X to L_{n-2}, and whose second component is the 3-cell filling the box once that face is chosen. Cubical Kan filling computes this horn-extension term from the displayed boundary data: hcomp synthesizes the missing face and hfill the accompanying filler. Thus the comparison with L_{n-2} factors through the already exported signature of L_n together with the adjacent bridge carried by L_{n-1}; it does not survive as a new primitive remote field in a μ-minimal presentation.

The same argument now iterates. Suppose inductively that the comparison from X to L_{n-r+1} has already been factored through the recent history. Since L_{n-r+1} was sealed against L_{n-r} and L_{n-r-1}, its exported trace again contains the adjacent unary bridge to L_{n-r} together with the corresponding binary square witnessing the comparison of composites. Combining that previously exported square with the already factored comparison for X forms another open 3-box, so one obtains another horn-extension fiber over already available boundary data. Cubical Kan filling computes that fiber again, yielding a comparison from X to L_{n-r} which factors through the normalized public signature of L_{n-r+1}. Inductively, every remote binary comparison is therefore a definitional consequence of the trace data already exported by the recent history, and 4.10 removes it from any μ-minimal signature as a primitive field. □

Corollary 4.13 (Two-layer chronological window)**.** *In the fixed fully coupled cubical extension calculus, every sealing step at stage $n+1$ has a genuine chronological window of size 2: all primitive obligations are generated by the exported traces of L_n and L_{n-1}, while comparisons with layers L_{n-r} for $r \geq 2$ factor through those two normalized signatures and are then synthesized by iterated Kan filling. Equivalently, the pair (L_n, L_{n-1}) forms a chronological Markov blanket for subsequent sealing.*

Proof. 4.11 shows the exact obligation-set stabilization $\mathcal{O}^{(k)}(X) \simeq \mathcal{O}^{(2)}(X)$ for all $k \geq 2$. 4.12 upgrades that dimensional statement to a temporal one: every surviving binary clause factors through the normalized signatures of the last two layers, so remote history contributes only boundary data already mediated by those exports. Hence primitive sealing data are exhausted by $S(L_n) \uplus S(L_{n-1})$, and all deeper history contributes only derived fillers. □

4.4.2. *Lower bound: explicit binary obstruction ($d \geq 2$).*

Theorem 4.14 (Explicit binary sealing obstruction)**.** *Let the active interface of a fully coupled univalent foundation export the two-point type* $\mathbf{2}$ *and the univalent path*

$$p_{\mathsf{swap}} := \mathsf{ua}(\mathsf{swap}) : \mathbf{2} \equiv \mathbf{2}.$$

Then there exists a globally acting promoted interface candidate whose unary clauses do not determine its sealing: sealing induces a genuine binary obligation over p_{swap}. Consequently the coherence depth is at least 2.

Proof. Consider the promoted interface candidate X whose advertised payload is a global endomap package

$$f : \Pi_{A:\mathcal{U}} A \to A.$$

At the current sealing stage, primitive clauses need only be supplied against the active exported interface. To specify one unary clause of this advertised action, choose

$$f_{\mathbf{2}} := \lambda_.\mathsf{left} : \mathbf{2} \to \mathbf{2}.$$

Because X is advertised as globally acting, sealing it against the active interface requires coherence not only on objects but also along exported paths in $\mathcal{U}$. The path p_{swap} therefore induces the binary obligation

$$\alpha_{\mathsf{swap}} : \mathsf{transport}_{\lambda A.\, A \to A}(p_{\mathsf{swap}}, f_{\mathbf{2}}) \equiv f_{\mathbf{2}}.$$

Univalence computes transport in the endomorphism family by conjugation with swap, so

$$\mathsf{transport}_{\lambda A.\, A \to A}(p_{\mathsf{swap}}, f_{\mathbf{2}}) = \mathsf{swap} \circ f_{\mathbf{2}} \circ \mathsf{swap}^{-1} = \lambda_.\mathsf{right}.$$

Hence α_{swap} is exactly the type

$$(\lambda_.\mathsf{right}) \equiv (\lambda_.\mathsf{left}),$$

which is impossible because evaluation at left would force $\mathsf{right} = \mathsf{left}$. Thus the passage from depth 1 to depth 2 introduces a genuine new sealing obligation type: the unary clause $f_{\mathbf{2}}$ does not determine whether X is admissible, because the typechecker must still ask for the binary witness α_{swap}.

Therefore $\mathcal{O}^{(2)}(X)$ contains a binary obligation not already present in $\mathcal{O}^{(1)}(X)$, so the ambient foundation cannot have coherence depth 1. Equivalently, univalence exposes binary sealing sites that are invisible to unary data alone. This is the same obstruction formalized in `Metatheory/AdjunctionBarrier.agda` by the nontriviality of `swap-path` and the contradiction theorem `depth1-insufficient`. Hence $d \geq 2$. □

Remark 4.15 (Adjunction as categorical reading)**.** The triangle identities of an adjunction exhibit the same binary pattern at a categorical level: once the object part and the unit/counit data have been fixed, the remaining triangle coherence compares composites already determined at arity 1. The paper uses adjunctions in that explanatory role only. The internal lower-bound theorem of the extension calculus is the swap obstruction above.

4.4.3. *Topological exact family: clutching constructions.*

Theorem 4.16 (Clutching family)**.** *The HIT clutching presentation over S^2 furnishes an exact depth-2 witness: a genuine binary comparison is necessary to specify the bundle data, but no independent primitive arity-3 obligation occurs.*

Proof. We translate the clutching construction into higher-inductive syntax. In homotopy type theory, the 2-sphere is the suspension $S^2 := \Sigma S^1$, generated by point constructors

$$\mathsf{N}, \mathsf{S} : S^2$$

and a path constructor

$$\mathsf{merid} : S^1 \to (\mathsf{N} \equiv \mathsf{S})$$

[Uni13, CHM18]. To specify an exported layer whose total space is a principal bundle over S^2, it suffices to define a type family

$$P : S^2 \to \mathcal{U}.$$

By the dependent eliminator for the suspension, such a family is determined by exactly three pieces of data:

$$P_{\mathsf{N}} : \mathcal{U}, \qquad P_{\mathsf{S}} : \mathcal{U}, \qquad \gamma : \Pi_{x:S^1}\big(P_{\mathsf{N}} \equiv P_{\mathsf{S}}\big).$$

By univalence, the last component may equally be read as a family of equivalences between the two pole fibers, which is the type-theoretic clutching datum.

Now S^1 is itself a higher inductive type generated by a point base and a loop $\mathsf{loop} : \mathsf{base} \equiv \mathsf{base}$. Therefore giving γ requires exactly two layers of information: first a base equivalence

$$\gamma(\mathsf{base}) : P_{\mathsf{N}} \equiv P_{\mathsf{S}},$$

which is unary data, and second a path over loop identifying the two transports induced by that equivalence around the circle. That second datum is a genuine binary coherence cell: it compares two unary composites and cannot be recovered from the endpoints alone. Hence depth 1 does not suffice.

Crucially, the HIT signature of S^2 contains no constructor of dimension 3 or higher. Accordingly, the elimination rule for S^2 asks for no primitive input beyond the two point data and the path family γ above. Once the binary path-over-loop has been supplied, any further compatibility condition appears only as a horn-extension problem over the already displayed eliminator boundary. By the same cubical filling argument used above, that higher extension type is computed from the boundary and therefore disappears from every μ-minimal public presentation by 4.10. Thus the clutching presentation furnishes an exact depth-2 witness. □

Corollary 4.17. *In the fixed fully coupled cubical extension calculus considered above, the coherence depth is exactly* 2.

Proof. 4.11 provides the upper bound by showing the exact obligation-set equivalence $\mathcal{O}^{(k)}(X) \simeq \mathcal{O}^{(2)}(X)$ for all $k \geq 2$, and 4.10 turns that equivalence into the syntactic statement that no μ-minimal public signature needs primitive arity-3 data. 4.12 shows that this depth-two upper bound is realized by a genuine two-layer chronological window rather than merely by a bound on cell dimension. 4.14 and 4.16 provide lower bounds by exhibiting genuine depth-two phenomena that do not collapse in the univalent setting. Therefore the least stabilizing depth is 2. □

5. The complexity scaling theorem

Let e_n be the admissible surface extension declaration whose sealing exports the nth sealed layer L_n, and write

$$\mu_n := \mu(e_n)$$

for its minimal opaque cost, and let κ_n be its core payload size. By 3.5, μ_n is exactly the abstract sealing cost $\Delta(X(e_n) \mid \mathcal{B}_{n-1})$, but it is phrased on canonical normalized public signatures and is therefore presentation-invariant by construction. The recurrence theorem therefore uses not only the stabilization statement of 4.6 but also a theorem identifying a genuine chronological window. In the depth-two cubical univalent case, this is exactly 4.13; in the UIP case, the proof of 4.8 shows that all non-unary comparisons collapse to their boundary, so the window is one layer. More generally, whenever a foundation has coherence depth d realized by a chronological window of size d, the primitive active basis interface at stage $n+1$ is the depth-d historical basis built from the preceding layers:

$$I^{(d)}_{n,\mathrm{bas}} = \biguplus_{j=0}^{d-1} S_{\mathrm{bas}}(L_{n-j}).$$

Thus the recurrence counts the chosen irreducible basis of the active interface rather than the raw number of exported generators. For the initial bootstrap stages before the full

window has been populated, we adopt the natural convention that nonexistent earlier layers contribute the empty interface.

5.1. **The universal affine recurrence.**

Theorem 5.1 (Universal affine recurrence for minimal opaque cost)**.** *For every foundation whose coherence depth d is realized by a chronological window of size d, the minimal opaque cost satisfies*

$$\mu_{n+1} = \sum_{j=0}^{d-1} \big(\mu_{n-j} + \kappa_{n-j}\big) \qquad (n \geq d).$$

Proof. By 3.5, the next minimal opaque cost is the abstract sealing cost, so 2.21 identifies it with the size of the active basis interface:

$$\mu_{n+1} = |I^{(d)}_{n,\mathrm{bas}}|.$$

Using the tagged coproduct decomposition of 2.16,

$$|I^{(d)}_{n,\mathrm{bas}}| = \sum_{j=0}^{d-1} |S_{\mathrm{bas}}(L_{n-j})|.$$

Now apply 3.5, equivalently the Integration Trace Principle 2.22, which gives

$$|S_{\mathrm{bas}}(L_{n-j})| = \mu_{n-j} + \kappa_{n-j}$$

for each j. Substituting yields the claimed affine recurrence. □

For later comparison, write

$$\tau_n := \sum_{i=1}^{n} \mu_i$$

for the cumulative minimal opaque cost through stage n.

5.2. **Affine one-step growth at $d = 1$.**

Corollary 5.2. *If the coherence depth is 1, this depth is realized by a one-layer chronological window, and every layer contributes the same core payload size c, then*

$$\mu_{n+1} = \mu_n + c.$$

If moreover the initial state exports no prior sealed layers, then

$$\mu_1 = 0, \qquad \mu_n = (n-1)c,$$

and therefore

$$\tau_n = c\,\frac{n(n-1)}{2}.$$

Minimal opaque cost therefore grows linearly. The cumulative opaque cost grows quadratically.

Proof. Substitute $d = 1$ into 5.1. Then only the term $j = 0$ remains, so

$$\mu_{n+1} = \mu_n + \kappa_n.$$

Under the uniform payload assumption $\kappa_n = c$, this becomes $\mu_{n+1} = \mu_n + c$. If the initial state exports no prior sealed layers, then the first sealing step sees the empty interface and therefore has minimal opaque cost $\mu_1 = 0$. Induction gives $\mu_n = (n-1)c$, and summing this arithmetic progression yields the stated formula for τ_n. □

5.3. **Affine Fibonacci scaling at $d = 2$.** By 4.13, the fixed cubical depth-two theorem is chronologically realized by the two most recent sealed traces.

Assume in this subsection that every sealed layer contributes the same core payload size c, and write $(F_n)_{n\geq 1}$ for the Fibonacci sequence with $F_1 = F_2 = 1$.

Corollary 5.3 (Affine Fibonacci recurrence)**.** *In the fixed fully coupled cubical extension calculus, if the coherence depth is 2 and every sealed layer contributes core payload size c, then*

$$\mu_{n+1} = \mu_n + \mu_{n-1} + 2c.$$

If we define

$$U_n := \mu_n + 2c,$$

then

$$U_{n+1} = U_n + U_{n-1}.$$

If moreover the initial state exports no prior sealed layers, then

$$\mu_1 = 0, \qquad \mu_2 = c,$$

so

$$U_n = cF_{n+2}, \qquad \mu_n = c(F_{n+2} - 2),$$

and the cumulative opaque cost satisfies

$$\tau_n = c(F_{n+4} - 2n - 3).$$

Proof. The recurrence is the case $d = 2$ of 5.1 together with the uniform payload assumption $\kappa_n = c$, giving

$$\mu_{n+1} = \mu_n + \mu_{n-1} + 2c.$$

Adding $2c$ to both sides yields

$$U_{n+1} = U_n + U_{n-1}.$$

If the initial state exports no prior sealed layers, then the first layer has no earlier interface against which to cohere, so $\mu_1 = 0$. The second layer sees only the first layer's export, whose size is $\kappa_1 + \mu_1 = c$, hence $\mu_2 = c$. Therefore

$$U_1 = 2c = cF_3, \qquad U_2 = 3c = cF_4.$$

Since both (U_n) and (cF_{n+2}) satisfy the same Fibonacci recurrence and agree at $n = 1, 2$, induction gives $U_n = cF_{n+2}$. Subtracting $2c$ yields

$$\mu_n = c(F_{n+2} - 2).$$

Summing from $i = 1$ to n gives

$$\tau_n = c\sum_{i=1}^{n}(F_{i+2} - 2) = c(F_{n+4} - 2n - 3),$$

using the standard Fibonacci summation identity. □

Corollary 5.4 (Asymptotic inflation factor)**.** *Under the bootstrap and uniform-payload hypotheses of 5.3, the structural inflation factor*

$$\Phi_n := \frac{\mu_n}{\mu_{n-1}}$$

converges to the Golden Ratio

$$\varphi = \frac{1+\sqrt{5}}{2}.$$

Proof. Under the bootstrap assumptions of 5.3,

$$\Phi_n = \frac{\mu_n}{\mu_{n-1}} = \frac{F_{n+2}-2}{F_{n+1}-2}.$$

The additive constants are asymptotically negligible relative to the Fibonacci growth, so the standard ratio convergence gives $\Phi_n \to \varphi$. □

5.4. **Metatheoretic robustness and universality across 2D foundations.** The counting notions used above are not source-level line counts. They are dimensions of the sealed typing judgment after passage to the counting normal form of 2.6. The first corollary records that robustness explicitly.

Corollary 5.5 (Robustness under admissible refactoring)**.** *Let two presentations of the same sealed payload or obligation judgment be related by canonical telescope isomorphism: rebundling or splitting dependent records, reassociation of Σ-types, currying or uncurrying iterated function arguments, or transparent internal normalization that does not change which fields are primitively inhabited. Then the induced sets of atomic payload schemas, atomic obligations, and historical supports are canonically bijective. In particular, the quantities $\kappa(X)$, $\Delta(X \mid \mathcal{B})$, and $a(o)$ are unchanged, and for surface declarations the induced minimal opaque cost $\mu(e)$ is unchanged as well.*

Proof. All of the listed refactorings preserve the same dependent typing judgment and differ only by canonical rebracketing or transparent elaboration of its telescope. After flattening the Σ-spine and discarding uniformly synthesized fields, both presentations reduce to the same counting normal form of 2.6 and 4.1. The atomic payload generators, atomic obligations, and their supporting layer indices therefore correspond bijectively. Since the paper's metrics are cardinalities of those normal-form sets, they are invariant under such refactorings. The corresponding trace subfamilies of normalized public signatures are therefore canonically bijective as well, so 3.4 gives the invariance of μ. □

The cubical depth-two theorem can now be separated from the particular implementation mechanism used to prove it. What matters abstractly is a lower-bound source of genuine binary coherence, an exact stabilization theorem for obligation sets at depth 2, and a factorization theorem forcing all surviving binary data through the previous two normalized signatures. A separate presentation-invariant elimination theorem for primitive arity-3 and higher trace fields may then be added as a surface-syntactic corollary, but it is not part of the abstract exact-depth wrapper below.

Theorem 5.6 (Depth-two law for 2D foundations)**.** *Let a fully coupled foundation satisfy the following two properties.*

(1) *There exists a candidate X whose normalized binary obligation set is not canonically equivalent to its unary one:*

$$\mathcal{O}^{(2)}(X) \not\simeq \mathcal{O}^{(1)}(X).$$

(2) *For every candidate X and every $k \geq 2$, there is a canonical equivalence*

$$\mathcal{O}^{(k)}(X) \simeq \mathcal{O}^{(2)}(X),$$

and every surviving binary clause factors through a chronological window of size 2.

Then the foundation has coherence depth 2. Under the uniform payload hypothesis $\kappa_n = c$, its minimal opaque cost satisfies

$$\mu_{n+1} = \mu_n + \mu_{n-1} + 2c,$$

so the shifted sequence $U_n := \mu_n + 2c$ is Fibonacci.

Proof. Property (2) gives exact stabilization $\mathcal{O}^{(k)}(X) \simeq \mathcal{O}^{(2)}(X)$ for all $k \geq 2$ together with a genuine two-layer chronological window. Hence 4.6 and 5.1 imply that the depth is at most 2 and that the recurrence is the depth-two affine law. Property (1) excludes depth 1. Hence the least stabilizing depth is exactly 2, and substituting $d = 2$ and $\kappa_n = c$ into 5.1 gives the displayed formula. The Fibonacci shift is exactly the argument of 5.3. □

Cubical type theory realizes Property (2) by Kan composition together with recent-history factorization. Two-level type theory suggests a different possible implementation pattern: the fibrant equality retains genuine higher path structure, while the strict outer equality supplies a separate level in which higher coherence bookkeeping can be strictified without reintroducing new primitive fibrant trace fields [ACK16, ACKS23]. The present paper does not claim that plain HoTT, or every 2LTT presentation, automatically instantiates 5.6. What the theorem isolates is only the hypothesis pattern: once a foundation has genuine binary coherence, exact stabilization of obligation sets at depth 2, and a two-layer chronological window, the same depth-two affine law follows.

6. Cubical Agda mechanization

6.1. **Mechanizing the theorem package.** The formalization is organized around the fixed extension calculus of section 2, its depth theorems, the induced recurrence law, and the clutching construction. Table 1 first makes the trusted boundary local. Here *fully formalized* means that the paper's statement is realized directly in the cited Agda modules; *formalized abstractly, read through bridge* means that the Agda development proves the structural theorem and the paper reads it through the normalization and canonical-presentation results of section 3; and *paper-level only* marks the bridge layer itself. The itemized map after the table then records the exact paper-to-Agda theorem correspondence.

- 2.12 corresponds exactly to `Metatheory/InterfaceCalculus.agda` via `transparent-growth-keeps-library-state`, `transparent-definitions-preserve-active-interface`, `transparent-definitions-have-zero-integration-l` and `transparent-user-level-code-lies-outside-the-recurrence`.

Table 1: Mechanization status of the main theorem stack.

Paper result	Formal status	Agda module(s)	Bridge?
section 3 normalization package	paper-level only	none	n/a
2.12	fully formalized	`Metatheory/InterfaceCalculus.agda`	no
2.21, 2.22, 3.5	formalized abstractly, read through bridge	`Metatheory/CanonicityDensity.agda`, `Metatheory/TracePrinciple.agda`, `Metatheory/InterfaceCalculus.agda`	yes
4.5, 4.8	fully formalized	`Metatheory/Obligations.agda`, `Metatheory/Extensional.agda`	no
4.9, 4.11, 4.13	fully formalized	`Metatheory/KanSubsumption.agda`, `Metatheory/UpperBound.agda`, `Metatheory/ChronologicalWindow.agda`	no
4.10, 4.12	formalized abstractly, read through bridge	`Metatheory/KanSubsumption.agda`, `Metatheory/UpperBound.agda`, `Metatheory/ChronologicalWindow.agda`	yes
4.14, 4.16, 4.17	fully formalized	`Metatheory/AdjunctionBarrier.agda`, `Geometry/Clutching.agda`, `Metatheory/ExactDepth.agda`	no
5.6	fully formalized	`Metatheory/TwoDFoundations.agda`	no
5.1, 5.2, 5.3	formalized abstractly, read through bridge	`Metatheory/UniversalRecurrence.agda`, `Core/DepthOneAffine.agda`, `Core/AffineRecurrence.agda`	yes
5.5	fully formalized	`Metatheory/Refactoring.agda`	no

- 2.21 is the paper's basis-sensitive reformulation of `Metatheory/CanonicityDensity.agda` via `CanonicityDensityTheorem` and `global-admissibility-forces-maximal-interface-density`, combined with the bridge-level normalization and canonical-presentation results of section 3.
- 2.22 corresponds exactly to `Metatheory/TracePrinciple.agda` via `IntegrationTracePrinciple`, `public-counting-normal-form`, and `integration-trace-principle`, built on the explicit sealed-layer presentation `ExplicitSealedLayer` from `Metatheory/InterfaceCalculus.agda`.
- 3.5 is the paper's bridge corollary combining 2.21, 2.22, and the normalization/canonical-presentation results of section 3; the Agda development formalizes the abstract counting theorems, while the passage to the syntactic quantity μ is the paper's normalized-signature reading of those results.

- 4.5 corresponds exactly to `Metatheory/Obligations.agda` via `historical-arity-forces-cell-dimension` and `irreducible-obligation-requires-cell`.
- 4.8 corresponds exactly to `Metatheory/Extensional.agda` via `UIP-forces-depth-1` and `history-truncates-to-on`
- 4.9 corresponds exactly to `Metatheory/KanSubsumption.agda` via `HornExtensionFiber`, `structural-integration-h` and `remote-layer-obligation-derived`.
- 4.10 is the paper's syntactic corollary of `Metatheory/KanSubsumption.agda` and `Metatheory/UpperBound.agda`: the Agda development formalizes the horn-extension fiber, its canonical inhabitant, and stabilization at depth 2 via `HornExtensionFiber`, `structural-integration-horn-reduction`, `remote-layer-obligation-derived`, `structural-obligation-set-equivalence`, and `structural-stabilizes-at` the elimination of primitive arity-3 fields from μ-minimal public signatures is obtained in the paper by combining those results with the bridge normalization theorems of section 3.
- 4.12 corresponds exactly to `Metatheory/KanSubsumption.agda` via `TelescopicSubsumptionView`, `telescopic-subsumption`, and `telescopic-remote-comparison-derived`; its reading as recent-history factorization through normalized public signatures is again the paper's bridge-level interpretation of that formal trace chain.
- 4.11 corresponds exactly to `Metatheory/UpperBound.agda` via `structural-obligation-set-equivalence` and `structural-stabilizes-at-two`.
- 4.13 corresponds exactly to `Metatheory/ChronologicalWindow.agda` via `primitive-obligations-factor-through-two-layer-chronological-window`, and `chronological-markov-blanket`.
- 4.14 corresponds exactly to `Metatheory/AdjunctionBarrier.agda` via `explicit-binary-sealing-obstruction`.
- 4.15 corresponds to the same module's categorical wrapper via `triangle-identity-corollary` and `adjunction-barrier`.
- 4.16 corresponds exactly to `Geometry/Clutching.agda` via `clutching-family-theorem`, `hopf-binary-clutching-n` and `clutching-horn-extension-fiber-contractible`.
- 4.17 corresponds exactly to `Metatheory/ExactDepth.agda` via `structural-coherence-depth-exactly-two`, `structural-chronological-window-size-exactly-two`, `cubical-coherence-depth-exactly-two`, and `cubical-chronological-window-size-exactly-two`.
- 5.6 corresponds exactly to `Metatheory/TwoDFoundations.agda` via `depth-two-law-for-2d-foundations`, `chronological-window-size-two-for-2d-foundations`, `constant-payload-depth-two-law`, `cubical-depth-two-law-for-2d-foundations`, and `cubical-chronological-window-size-two-for-2d-foundat`
- 5.1 corresponds exactly to `Metatheory/UniversalRecurrence.agda` via `UniversalAffineRecurrence` and `universal-affine-recurrence`, read through 3.5 so that the paper's surface-syntactic recurrence is stated over μ.
- 5.2 corresponds exactly to `Core/DepthOneAffine.agda` via `depth1-affine-growth`, `Delta-depth1-closed`, and `tau-depth1-closed`; in the paper these arithmetic laws are read through 3.5 as statements about minimal opaque cost.
- 5.3 corresponds exactly to `Core/AffineRecurrence.agda` via `Delta-bootstrap`, `U-bootstrap-closed`, and `tau-bootstrap-closed`; again the code's `Delta`-named arithmetic sequence is the paper's μ-sequence after the bridge identification of 3.5.
- 5.5 corresponds exactly to `Metatheory/Refactoring.agda` via `historical-support-correspondence` and `refactoring-invariance`.

No module cited in the theorem map above uses `postulate`. Postulates do occur in auxiliary modules elsewhere in the code base, but not in the modules invoked here.

The trusted base is Agda with Cubical support together with the Cubical library declared in `pen.agda-lib`. The arithmetic layer uses `Core/Nat.agda`; the clutching formalization additionally uses the Cubical library's standard higher-inductive circle and suspension modules `Cubical.HITs.S1.Base` and `Cubical.HITs.Susp.Base`, together with the standard equivalence, isomorphism, and univalence infrastructure.

The formalization includes both direct cubical metatheory and a distilled extension calculus. The UIP collapse, horn-extension computation, recent-history factorization chain,

lower-bound obstruction, exact depth-two corollary, and clutching-family witness are mechanized against cubical or UIP primitives themselves. The surface-syntactic theorems about normalized signatures and minimal opaque cost are then obtained in the paper by combining those formal metatheorems with the bridge normalization results of section 3. The canonicity, trace, refactoring, recurrence, and abstract 2D-foundations layers package those results as the sealed-interface and counting calculus studied in the paper. The formalized core is therefore explicit, but the passage to the paper's μ-based syntax still uses the bridge.

6.2. **Barrier discipline and affine recurrence.** Its organizing invariant is that resolved layers are exported through opaque interfaces. At the level of proof architecture, this is the exact content of 2.22, realized in Agda by `integration-trace-principle`: once an obligation has been discharged, later constructions depend only on the exported schema and not on the hidden derivation tree that discharged it. In the barrier modules, this is checked locally for the transitions around steps 8 and 9 by showing that inherited obligations are discharged from the owning layer's opaque record interface alone, without reopening deeper internal derivations.

This realizes the abstraction barrier promised by 2.8. In the formalization, 2.12 is exactly the package `transparent-growth-keeps-library-state`, `transparent-definitions-preserve-active` `transparent-definitions-have-zero-integration-latency`, and `transparent-user-level-code-lies-o` It is the reason historical depth is measured in terms of sealed layers rather than in terms of raw derivation ancestry.

On the counting side, the Agda development makes the modeling discipline explicit. The active primitive interface is represented as the disjoint union of the two most recent schema sets, and the saturation step is stated separately rather than hidden in the syntax of imports. In the paper's current basis-sensitive presentation, recurrence counts the corresponding irreducible basis sites of that interface and then reads that abstract count as minimal opaque cost via 3.5. The concrete decompositions in `Saturation/Decomposition.agda` then record, step by step, how the next obligation set splits into contributions from the two most recent sealed layers.

`Metatheory/UniversalRecurrence.agda` proves `universal-affine-recurrence`, while `Core/DepthOneAffine.agda` and `Core/AffineRecurrence.agda` package the exact arithmetic specializations `depth1-affine-growth`, `Delta-depth1-closed`, `tau-depth1-closed`, `Delta-bootstrap`, `U-bootstrap-closed`, and `tau-bootstrap-closed`. In particular, `Core/AffineRecurrence.agda` extends the payload-free counting law to the payload-aware recurrence

$$\mu_{n+1} = \mu_n + \mu_{n-1} + 2c,$$

defines the shifted sequence $U_n = \mu_n + 2c$, and proves that U satisfies the homogeneous Fibonacci recurrence. In the code base those arithmetic lemmas still carry `Delta`-names, because they are formalized on the abstract counting sequence before the bridge identification with μ. The zero-payload specialization is linked back to `Core/Nat.agda`, while `ObligationGraph/Recurrence.agda` records the payload-free corollary and `Test/Fibonacci.agda` checks the concrete Fibonacci identities.

The mechanization separates four claims that are easy to conflate in prose. The *abstraction-barrier claim* says that historical memory is mediated only by opaque exports. The *metatheoretic depth claim* says that cubical filling uniformly derives witnesses for arity-3 obligations while binary obligations remain genuinely independent in general. The *syntactic minimality*

claim says that, after passage through the bridge normalization theorems, those metatheoretic facts eliminate primitive arity-3 fields from μ-minimal public signatures and force recent-history factorization of the surviving binary ones. The *recurrence claim* says that, once the depth-two window and explicit payload accounting are in place, the cardinality of the next primitive basis interface is governed by the affine depth-two law above. The formalization establishes the metatheoretic ingredients and the abstract counting law, and the paper records the bridge reading that turns them into the μ-based theorem stack.

Remark 6.1 (Scope of the mechanization)**.** The formalization establishes the cubical lower/upper metatheory, the abstract candidate-and-obligation calculus, and the recurrence package built on them. The exact paper-to-Agda correspondences are listed above; the cited modules use no `postulate`; and the clutching module is checked separately only because its HIT imports live on the Cubical library's standard natural-number interface rather than on `Core/Nat.agda`. The mathematical arguments of section 4 remain important as explanation and as links to the categorical and topological examples. What remains at the paper level is precisely the bridge from those formal metatheorems to the normalized-signature quantity μ, including the surface-syntactic readings of 4.10, 4.12, and the μ-based recurrence.

7. Conclusion

The paper isolates *coherence depth* as a metatheoretic invariant of a fixed cubical extension calculus. The associated payload counts, abstract latency counts, and normalized-signature quantity μ are taken on counting normal forms of sealed interfaces, so they are stable under admissible record rebundling, interface slicing, and other canonical telescope refactorings. In extensional settings with UIP, higher coherences collapse and the invariant is 1. In the fixed fully coupled cubical extension calculus studied here, the invariant is 2. The main metatheorem is therefore a depth-two affine recurrence for minimal opaque cost; under uniform payload size, the shifted sequence is Fibonacci.

The mechanism behind the recurrence is structural. In the formalization, 2.21 is read through `global-admissibility-forces-maximal-interface-density` together with the bridge-level normalization theorems, 2.22 by `integration-trace-principle`, and 4.13 by `two-layer-chronological-window`. Sealing cost is therefore identified with the size of the active basis interface, while resolved coherence data disappear as derivations but remain present in the public signature as trace fields. In the cubical case, the marginal higher-arity data are horn extensions rather than closed-sphere fillers; `structural-stabilizes-at-two` and `cubical-coherence-depth-exactly-two` make that upper statement exact, 4.10 turns it into the statement that primitive arity-3 fields disappear from μ-minimal public signatures, and `primitive-obligations-factor-through-last-two` gives the recent-history factorization underlying the two-layer window. The explicit swap obstruction and the clutching family supply exact lower-bound witnesses. Under constant payload size `universal-affine-recurrence`, read through 3.5, specializes to the affine depth-two law and `Delta-bootstrap` gives the shifted Fibonacci corollary.

From this viewpoint the appearance of the Golden Ratio is not an empirical regularity of one implementation and not a matter of aesthetic numerology. It is the characteristic root of the homogeneous recurrence recovered after separating the constant payload contribution from the depth-two coherence memory.

The Cubical Agda formalization therefore certifies the cubical metatheory and abstract recurrence package on which the paper's extension-calculus theorem stack rests, rather than merely illustrating isolated ingredients. section 6 records the exact paper-to-Agda map; representative formal counterparts are `history-truncates-to-one` for the UIP collapse, `structural-stabilizes-at-two` and `cubical-coherence-depth-exactly-two` for the exact depth package, `primitive-obligations-factor-through-last-two` for the recent-history factorization core, `cubical-depth-two-law-for-2d-foundations` for the abstract 2D-foundations wrapper, and `Delta-bootstrap`, read through 3.5, for the shifted Fibonacci corollary. The trust assumptions are exactly the Agda/Cubical kernel, the declared Cubical library, and the arithmetic layer used by the development, with no postulates in the cited proof modules; what remains outside the formal core is the bridge from those metatheorems to the paper's normalized-signature quantity μ. A natural next step is to connect this metatheory more tightly to concrete systems for generating and integrating candidate extensions.

References

[ACK16] Thorsten Altenkirch, Paolo Capriotti, and Nicolai Kraus. Extending homotopy type theory with strict equality. In *25th EACSL Annual Conference on Computer Science Logic (CSL 2016)*, volume 62 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 21:1–21:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2016. URL: `https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CSL.2016.21`, `doi:10.4230/LIPIcs.CSL.2016.21`.

[ACKS23] Danil Annenkov, Paolo Capriotti, Nicolai Kraus, and Christian Sattler. Two-level type theory and applications. *Mathematical Structures in Computer Science*, 33(7):688–743, 2023. `doi:10.1017/S0960129523000130`.

[AKGB24] Frederik Lerbjerg Aagaard, Magnus Baunsgaard Kristensen, Daniel Gratzer, and Lars Birkedal. Unifying cubical and multimodal type theory. *Logical Methods in Computer Science*, 20(4), 2024. URL: `https://lmcs.episciences.org/14929`, `doi:10.46298/lmcs-20(4:25)2024`.

[BCM+20] Lars Birkedal, Ranald Clouston, Bassel Mannaa, Rasmus Ejlers Møgelberg, Andrew M. Pitts, and Bas Spitters. Modal dependent type theory and dependent right adjoints. *Mathematical Structures in Computer Science*, 30(2):118–138, 2020. `doi:10.1017/S0960129519000197`.

[CCHM18] Cyril Cohen, Thierry Coquand, Simon Huber, and Anders Mörtberg. Cubical type theory: a constructive interpretation of the univalence axiom. In *21st International Conference on Types for Proofs and Programs (TYPES 2015)*, 2018.

[CHM18] Thierry Coquand, Simon Huber, and Anders Mörtberg. On higher inductive types in cubical type theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*, 2018.

[Hus94] Dale Husemoller. *Fiber Bundles*, volume 20 of *Graduate Texts in Mathematics*. Springer, 3 edition, 1994.

[KvR19] Nicolai Kraus and Jakob von Raumer. Coherence via well-foundedness. In *Proceedings of the 34th Annual ACM/IEEE Symposium on Logic in Computer Science*, 2019.

[Lan63] Saunders Mac Lane. Natural associativity and commutativity. *Rice University Studies*, 49(4):28–46, 1963.

[Lum10] Peter LeFanu Lumsdaine. Weak ω-categories from intensional type theory. In *Typed Lambda Calculi and Applications*, volume 6690 of *Lecture Notes in Computer Science*, pages 172–187, 2010.

[Sta63] James D. Stasheff. Homotopy associativity of H-spaces I, II. *Transactions of the American Mathematical Society*, 108(2):275–312, 1963.

[Uni13] Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013. URL: `https://homotopytypetheory.org/book`.

[vdBG11] Benno van den Berg and Richard Garner. Types are weak ω-groupoids. *Proceedings of the London Mathematical Society*, 102(2):370–394, 2011.

[VMA19] Andrea Vezzosi, Anders Mörtberg, and Andreas Abel. Cubical agda: a dependently typed programming language with univalence and higher inductive types. In *Proceedings of the 24th ACM SIGPLAN International Conference on Functional Programming*, 2019.